TORONS: Mapping the Unseen Web

1st Suryansh Deshwal
Department of CSE(AI & ML)
Krishna Institute of Engineering & Technology (KIET)
Ghaziabad, Delhi-NCR, Uttar Pradesh, India
suryanshdeshwal@gmail.com

2nd Kanishka
Department of CSE
Krishna Institute of Engineering & Technology (KIET)
Ghaziabad, Delhi-NCR, Uttar Pradesh, India
kanishka.2327cse1011@kiet.edu

3rd Ambar Chakarvartty
Department of CSE
Krishna Institute of Engineering & Technology (KIET)
Ghaziabad, Delhi-NCR, Uttar Pradesh, India
ambar.2327cse1058@kiet.edu

4th Bhuvnesh Malik
Department of CSE(AI & ML)
Krishna Institute of Engineering & Technology (KIET)
Ghaziabad, Delhi-NCR, Uttar Pradesh, India
bhuvnesh.malik@kiet.edu

***Abstract*—The Dark Web, which consists of most of the internet, is composed of all the anonymous networks that hide the identity of the user and server, which allows threat actors to perform illegal and wrong activities. These anonymous networks comprise TOR, I2P and freenet, etc. TORONS is a dark web crawler, i.e. designed to efficiently extract and archive the content from both Tor and I2P, and more of these networks simultaneously, on the same machine with minimal resource utilization, providing some real-world features like web-archives (Dark web), Classified database for model training, custom Crawler engine, etc. Unlike other crawlers, TORONS uses a NLP-based hate speech detection algorithm that searches for specific hate keywords on different dark Web forums and creates a web archive of all crawled websites. Crawled data is stored in a MongoDB-based database, creating a web archive for the dark web. This data can later be used to train various AI models for a more perfect segregation of such dark web forums. This data can then be used for making practical tools for OSINT, cyber forensics and security research in anonymized ecosystems. The crawler uses a specialized technique to spawn and system level and physical threads and cores for parallel processing and asynchronous crawling on each particular process. It also supports CLI interface reducing the usage for resources from GUI and window managers, etc. It also has a modular structure which helps in using different devices to setup the different features for separate and secure processing. The crawler is entirely based on a CLI interface, offering efficient resource monitoring and suitable for different hardware conditions for various systems**

Keywords—Dark Web, Tor, I2P, Distributed Systems, Multithreading, NLP, Hate Speech Detection, Cybersecurity, Crawler

I. Introduction

The dark Web represents that portion of the Internet that remains hidden from traditional Internet users. It can only be accessed using specific protocols and browsers such as The Onion Router (Tor) and the Invisible Internet Project (I2P) protocols, etc [1], [2]. The onion and garlic structure used in these protocols helps users maintain anonymity while browsing different websites and forums on the dark web. Although these networks were originally designed to protect a user's privacy on the Internet, they are also frequently used for various illegal activities including the selling of abusive substances like drugs, human exploitation, money laundering, black market, etc [3], [4]. This concealed and protected environment presents unique challenges to cybersecurity researchers, law enforcement agencies and digital forensics who seek to monitor and analyze the dark web. Crawling such hidden services is not trivial as Websites often come and go without any proper notice, various sites require user registration, or limited repeated access via CAPTCHAs or anticrawling mechanisms. Also, the system resources required are quite high, as we can not use the GPU threads for parallel crawling as they do not support the network I/O, etc. Also, the high latency, the limited bandwidth, and the fragility of Anonymous networks make crawling very difficult, both in terms of resources and unreliability. Moreover, a meaningful analysis of the crawled content from the dark web requires human intelligence or advanced natural language processing, especially when attempting to detect threats like hate speech, terrorism recruitments, or illegal transactions. Although several dark web crawlers already exist, they often suffer from various limitations. Many are restricted to the Tor network and do not provide proper support on I2P networks as well as running any two networks simultaneously isn't practiced efficiently till now. Also, the limitations for parallelism in crawling using physical threads and controlling network I/O occur in almost all crawlers of this kind [5]. To overcome these challenges, we introduce TORONS - a lightweight, process-driven, distributed dark web crawler designed to operate seamlessly on both Tor and I2P like networks (more than 2 if required). TORONS is capable of handling login-based authentication bypass for various dark web forums and marketplaces, which helps to

979-8-3315-9576-0/26/$31.00 ©2026 IEEE

access deeper content. The crawler uses process-level parallelism and asynchronous I/O to maximize the overall hardware usage by the crawler, and overcome the bandwidth limitations. It provides a CLI-based interface enabling efficient monitoring of crawling tasks, system performance, and network status through integrated tools like NYX. In addition to the data collection, TORONS incorporates a hate speech detection module that performs basic content analysis during the crawling process and classifies the database into categories for further ease of training of the module. Likewise, using the same database, other nlp models and ML models can be run or trained on this database . This enables automated tagging and archiving of potentially harmful or illegal material on the dark web. The crawled data is stored in the MongoDB-powered database, and this database can be further used to train various machine learning models and the integration of those models in digital forensics and OSINT-based tools, etc. The system is container-ready, hardware-adaptive and suitable for deployment in both academic research and operational intelligence settings. In the following section, we review related work in dark web crawling and hate speech detection, its implementation and evaluation of TORONS in detail.

II. Related Work

A. Dark Web Crawling Advances (2022–2025)

c4darknet (2022) extends earlier c4i2p work to support Tor, I2P, and Freenet. It adds priority scheduling and retry cues, but lacks inline content analysis, leaving post-processing to external tools.

Marketplace-specific crawlers (2023) now benchmark Tor vs. I2P mirrors. A 2023 study found I2P sessions yield faster average response times (5.6 s vs. 9.1 s on Tor) and longer uninterrupted crawls when mirroring AlphaBay, highlighting reliability gaps between the two networks [6], [7].

CRATOR (2024) introduces a Tor-focused crawler that automates log-in, rotates user-agents and cookies, and outperforms ACHE in coverage and robustness when encountering CAPTCHAs and anti-bot hurdles [8].

Drawbacks noted by recent surveys include dependence on manual CAPTCHA solving, limited multi-network concurrency, and minimal integration with real-time analytics pipelines.

B. AI-Assisted Crawling & Threat Intelligence (2024–2025)

AI-Enhanced Dark Web Crawler (2024) couples reinforcement-learning-driven exploration with blockchain logging for tamper-evident telemetry. It emphasizes adaptivity to evolving marketplace structures but remains text-centric, omitting the multimedia extraction. Commercial platforms such as Webz.io expose unified APIs that continuously scrape both open and dark webs into structured feeds, yet operate as closed SaaS with opaque methods and costs.

C. Hate-Speech & Illicit-Content Detection Trends (2023–2025)

Transformer baselines dominate: Systematic reviews in 2023–2024 identify BERT and its derivatives as state-of-theart for textual hate speech, surpassing classic TF-IDF + SVM pipelines [9], [10]. Sentiment-Aided Hate Speech Detection (SAHSD, 2024) augments large-language-model outputs with sentiment cues, reducing false positives and highlighting the value of hybrid linguistic signals [11], [12]. Multi-modal detection is emerging: a 2025 framework fuses CNNs and LSTMs across text, images, audio, and video, achieving 98.5% accuracy on social-media datasets—indicating future dark-web crawlers will need to ingest more than HTML text.

D. TORONS in Context: Key Differences from Existing Crawlers

By identifying the limitations of previous frameworks, TORONS incorporate these features:

• Multi-Network Crawling: Few academic tools handle Tor and I2P crawling simultaneously, those that try (e.g., c4darknet) sacrifice real-time analysis. TORONS implements a hybrid multi-processing and asynchronous I/O model which allows spawning multiple crawler processes for Tor and I2P simultaneously [2], [13].

• Analysis Timing: Existing crawlers generally archive first, analyze later. TORONS embeds hate-speech detection during fetch, providing early triage.

• Concurrency Model: Existing crawlers such as CRATOR and c4darknet rely mainly on multi-threading or asynchronous event loops within a single process, which can limit CPU utilization for compute-heavy tasks. TORONS introduces a hybrid model that maps one process per physical CPU core, each running an asynchronous event loop. This approach combines true parallelism with non-blocking I/O, enabling better scalability for crawling and inline analysis.

III. Methodology

A. System Overview

TORONS is designed as a modular command-line interface application targeted at crawling, extracting, classifying, and archiving content from dark web platforms both on Tor and I2P networks. Its modularity enables updating or replacing its different components, such as the crawler engine, network modules, and the analysis pipeline, independently, which allows extensibility and maintainability. The TORONS framework has been designed to conform to the POSIX standard so that it can be executed on any Unix-like system. It is written in Python, with its core libraries used for process management, networking, and asynchronous task handling; thus, it is portable and efficient for large-scale deployment [1], [8].

B. Crawler Architecture

The architecture of the crawler relies on a strong producerconsumer paradigm. A thread-safe task queue manages the URLs that must be visited, which are then dynamically allocated to a pool of workers. Each of the workers is independent in terms of processing, fetching, and parsing content from the hidden services. The architecture has been designed to support HTTP and HTTPS traffic routed through SOCKS proxies, ensuring all communications traverse the anonymizing layers provided by Tor or I2P. In order to avoid common anticrawling

TABLE I

Comparative Analysis of TORONS with Existing Dark Web Crawlers

Feature	TORONS	CRATOR (2024)	ACHE	c4darknet (2022)
Network Support	Tor, I2P, and multiple darknets simultaneously	Primarily Tor network	Tor hidden services	Tor, I2P, and Freenet
Concurrency Model	Hybrid multi-process with asynchronous I/O mapped to physical cores	Breadth-first crawling with robust proxy handling	Focused crawling using page classifiers	Priority-based scheduling with retry queues
Analysis Timing	Inline, real-time analysis during content fetch	Post-processing after crawl completion	Post-processing with archive-first strategy	No inline content analysis
Security Bypassing	Automated login handling with session refresh	Advanced CAPTCHA and login mechanisms	Basic cookie and proxy support	Limited session preservation
Interface	POSIX-compliant CLI optimized for headless and constrained systems	Monolithic GUI or SaaS-based interface	REST API with web-based UI	Not explicitly specified

mechanisms, TORONS randomizes all HTTP headers, especially user-agent strings, and emulates browserlike request patterns, making its traffic less distinguishable from other legitimate visitors. This approach reduces the risk of detection and further blocking by target sites [2].

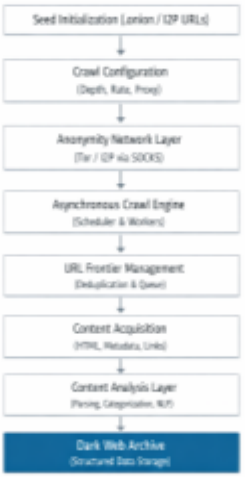

Fig 1: Workflow of the TORONS crawler architecture.

C. Multi-Process and Asynchronous Crawling

TORONS brings together both process-based parallelism and asynchronous I/O to achieve truly efficient crawling. By utilizing the multiprocessing library within Python, the system launches several worker processes, each bound to a physical CPU core. Inside every process, there is an event loop implemented using asyncio orchestrates many asynchronous tasks built with aiohttp. The combination of multiprocessing and asynchronous execution allows the TORONS to handle a vast number of the network requests simultaneously, since usually the communication over Tor and I2P networks is slow and sometimes unstable, the use of non-blocking I/O becomes essential. The asynchronous design ensures the crawler does not remains responsive and efficient even when some requests are delayed or fail, thus maximizing resource utilization and throughput [2].

D. Automated Login Handling

Scraping the dark web for meaningful data often requires bypassing authentication barriers. To do that, TORONS features an automated login module that can parse HTML forms and maintain user sessions. When possible, it can register disposable credentials, fill out form fields, and solve captchas, maintaining persistent session cookies across sites that allow public account creation. In environments where periodic session refresh or multi-factor authentication is required, the session state is monitored and recycled at will. All credential data and session artifacts are encrypted and stored temporarily in memory to minimize the risk of unauthorized exposure. The ability of TORONS to authenticate into protected forums and marketplaces greatly expands the coverage of crawling beyond that possible by traditional, unauthenticated crawlers [5].

E. Web Archiving with MongoDB

To enable efficient storage and future analysis, TORONS archives crawled content—including raw HTML, extracted text, and metadata—within a MongoDB database. Each record stores key attributes such as source URL, crawl timestamp, anonymizing network (Tor/I2P), session identifiers, and crawl status. Temporarily unreachable URLs are placed in a retry queue to mitigate the ephemeral nature

of dark web services. MongoDB's flexible document schema supports evolving content types without reconfiguration and enables efficient indexing and querying for downstream OSINT and machine learning workflows [7].

F. Hate Speech Detection Pipeline

TORONS includes an inline, rule-based text classification pipeline to identify illicit or hate-oriented content in real time. After a web page has been crawled, its textual content is parsed through BeautifulSoup, then filtered through a custom keyword-matching algorithm developed from curated datasets of known hate speech vocabularies. Each matching document automatically receives classification tags, stored with the original content in MongoDB for further analysis or alerting.

Given the modular nature of TORONS, this component could be replaced seamlessly with more advanced natural language processing models, such as transformer-based architectures (e.g., BERT), as either computational resources permit or needs evolve. This therefore allows investigators and analysts immediate insights into high-risk content but retains the flexibility to adopt future enhancements in machine-learning-driven classification [9], [14]. Overall, through the integration of advanced crawling, automation, data management, and real-time NLP analysis within one coherent platform, TORONS offers scalability and practicality within the collection and classification of intelligence invaluable in nature coming from dynamic and often unreachable parts of the dark web.

IV. Implementation

A. Tools and Technologies

Implementation of TORONS utilizes a set of open-source tools and libraries, selected to provide maximum flexibility and performance for a variety of tasks related to dark web crawling and content analysis:

• TOR and I2P Daemons: These are the anonymizing network services that form the basis of communication for TORONS to route any HTTP/HTTPS request over SOCKS proxies, which anonymize the source IP and open up access to hidden services available on the Tor and I2P networks.

• Python: Python is used as the primary implementation language due to its extensive ecosystem and suitability for network-intensive applications. TORONS employs requests and aiohttp for synchronous and asynchronous HTTP communication, BeautifulSoup for HTML parsing, and the multiprocessing and threading modules to support concurrent crawling and thread-safe task management.

• MongoDB / ElasticSearch: MongoDB serves as the main NoSQL database to which crawled pages and their metadata are archived; this is due to its flexibility in document-based schemas and horizontal scaling. Additionally, ElasticSearch can be integrated for more advanced full-text search capabilities that provide faster querying and complex analytics on large-scale archives.

• Transformers (Hugging Face BERT): NLP analysis is powered by transformer-based models; it uses the BERT architecture provided by Hugging Face's Transformers library, thus allowing contextual state-of-the-art text classification to detect hate speech in crawled data.

B. Code Workflow and Module Description

The system architecture is designed to separate the overall functionality into distinct, loosely coupled modules, thus further facilitating maintainability, testing, and future enhancements:

1) crawler.py: The main crawling engine: it handles the HTTP(S) request dispatching via Tor or I2P SOCKS proxies. It also implements network timeout and connection failure retries, and parses HTML responses using BeautifulSoup. The module takes care of link extraction for recursive crawling while respecting domain restrictions and politeness constraints.

2) queue_manager.py: This is responsible for distributing URLs via a thread-safe queue mechanism. It supports priority-based scheduling and retry logic for sites which may be temporarily unreachable to ensure a balanced workload across multiple crawling processes.

3) nlp_analyzer.py: This is a program that will run the BERT pipeline for hate speech classification on textual data extracted by crawler.py. It tokenizes and normalizes texts, runs a pretrained transformer model for inference, and saves classification results (labels and confidence scores) together with corresponding documents to the database [15].

4) main.py: It works as the orchestrator of the system and it is responsible for the initialization of the environment, spawning several crawler worker processes mapped to CPU cores, and the coordination of intermodule communication. It also handles persistent session management, task progress monitoring, and shutdown/recovery in case of a long-running crawl [2].

C. Additional Implementation Details Advanced techniques such as the following have been used in the implementation to attain maximum robustness and performance:

• Anonymization and Evasion: In every HTTP request, the User-Agent string is randomized, along with request headers, to make it look like different real browser clients for reducing fingerprinting risks. Time delays in requests are used to meet the target servers' rate limits and avoid detection as denial of service.

• Session and Credential Management: The automated login system securely manages authentication tokens and cookies within encrypted in-memory stores, performing periodic refreshing of credentials, where possible and including handling of CAPTCHA challenges to continue accessing restricted content.

• Concurrency Model: TORONS sustains high concurrency levels on network-bound operations common in accessing the dark web through the combination of multiprocessing and asynchronous I/O. Each process contains an asynchronous event loop to perform multiple concurrent requests without blocking, therefore optimizing the hardware.

• Data Persistence and Fault Tolerance: The crawling states, like the status of URLs and their metadata, are always persisted to MongoDB. The persistent state management enables graceful recoveries from crashes or unexpected shutdowns without losing data or redundancy in the crawling of TORONS [7].

Fig 2 . User interface of the TORONS system displaying real-time crawl progress, extracted links, and analysis summaries.

V. RESULT

A. Experimental Setup

The benchmarking of TORONS was performed on an OpenBSD system with the following specifications:

- Processor: 12-core, 16-thread x86 64 architecture CPU
- RAM: 16 GB
- Operating System: OpenBSD

The crawler was executed in three modes—Tor-only, I2P-only, and a combined mode crawling both simultaneously—for a duration of 5 minutes in each mode.

B. Benchmark Results

To evaluate the performance of TORONS, the crawler was tasked with extracting and storing URLs from hidden services on each network. Performance was assessed based on three key metrics: the number of URLs successfully stored versus failed, the crawling efficiency, and the crawl rate (URLs per second). The results from this benchmark are presented in Tables II–V.

TABLE II

TOTAL CRAWLED URLS FOR TORONS

Network	Total Crawled URLs
Tor	2428
I2P	1060
Both	4588

TABLE III

STORED VS. FAILED URLS FOR TORONS

Network	Stored URLs	Failed URLs
Tor	811	1617
I2P	120	940
Both	1663	2925

TABLE IV

CRAWLING EFFICIENCY OF TORONS

Network	Efficiency (%)
Tor	33.40
I2P	11.32
Both	36.24

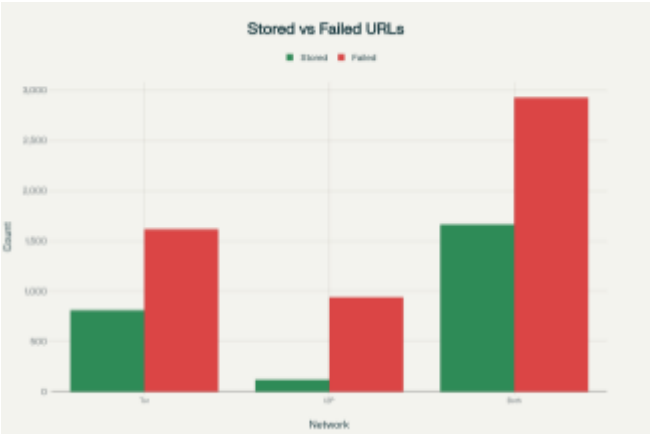

Fig 3 . Stored vs. Failed URLs for TORONS crawler across Tor, I2P, and combined networks. Total crawl time: 5 minutes.

TABLE V

CRAWL RATE FOR TORONS

Network	Crawl Rate (URLs/s)
Tor	8.09
I2P	3.53
Both	15.29

VI. DISCUSSION

Although TORONS shows big improvements in unified dark web crawling and real-time content analysis, there are some problems with the system and the environment it runs in that need to be thought about.

A. Dependence on Tor/I2P Network Uptime

The features and range of TORONS are closely linked to how well the anonymizing networks Tor and I2P work and how stable they are. Both networks are unstable by nature, with nodes frequently leaving and coming back, and services going down from time to time for maintenance or censorship. Because of this volatility, crawling cycles may not be complete, and data gaps may happen from time to time. TORONS has a way to try again to reach sites that are down, but if a network or service is down for a long time, you may lose access to some hidden resources for good. So, the system's effectiveness depends not only on how well it is built, but also on how healthy and easy it is to access these decentralized networks.

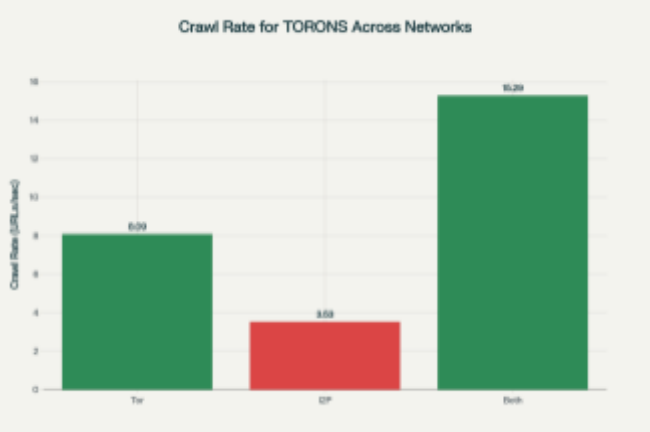

Fig 4 . Crawl Rate (URLs/second) for TORONS on Tor, I2P, and Combined networks, measured during a 5-minute benchmark.

B. False Positives in Hate Speech Detection

The hate speech detection pipeline is effective in identifying potentially harmful content but remains susceptible to false positives due to the inherent ambiguity of natural language on dark web forums. Factors such as sarcasm, slang, coded expressions, and dialectal variation often lead both keywordbased rules and transformer models to misclassify benign content or miss subtle contextual cues. Improving reliability will require continued model fine-tuning, incorporation of domain-specific lexicons, and hybrid approaches combining automated analysis with human-in-the-loop validation.

C. Challenges with Maintaining Long Sessions on Volatile Sites

Dark web services commonly enforce IP bans, session timeouts, and anti-crawling defenses such as throttling and CAPTCHAs, making long-lived authenticated sessions difficult to maintain. While TORONS mitigates some of these issues through automated login and session management, session volatility persists and can limit sustained access to restricted or dynamically gated content, resulting in partial data collection. Future enhancements include more adaptive session handling, improved crawl strategies, and tighter integration with CAPTCHA-solving techniques.

D. Broader Implications and Future Directions

Despite these limitations, TORONS represents meaningful progress toward scalable dark web intelligence gathering and forensic analysis. Many current constraints can be addressed through future work, including support for additional anonymizing networks, adoption of more robust crawling protocols, and refinement of NLP models to reduce classification errors within a modular system design. Deploying systems such as TORONS in real-world investigative settings also requires careful balancing of operational effectiveness with ethical and legal considerations. Continued research and collaboration with law enforcement and cybersecurity stakeholders will be essential to improve practical utility while minimizing unintended risks.

VII. Conclusion and Future Work

This paper presents TORONS, a modular and scalable framework for dark web crawling, which can efficiently extract, archive, and analyze hidden services operating on Tor and I2P networks. Equipped with multiprocessing asynchronous crawling, automated handling of logins, and inline NLPHate speech detection, TORONS effectively overcomes major hurdles to real-time intelligence collection within anonymous and dynamic ecosystems. Its MongoDB-supported web archive offers further support to downstream intelligence operations and digital forensic analysis.

Future work will focus on expanding TORONS' analytical and operational capabilities through active learning–based NLP adaptation, improved crawl scheduling and prioritization, and the integration of honeypot-driven mechanisms to gain deeper insight into malicious actors and behaviors. Overall, TORONS provides a strong foundation for an automated resilient dark web intelligence gathering tool, with potential for meaningful impact.

References

[1] M. K. Alshammery and A. F. Aljuboori, "Crawling and mining the dark web: A survey on existing and new approaches," Iraqi Journal of Science, vol. 63, no. 3, pp. 1339–1348, Mar. 2022.

[2] C. C. Aggarwal and D. Rajagopalan, "Hybrid focused crawling on the surface and the dark web," EURASIP Journal on Information Security, 2017.

[3] M. Zabihimayvan, R. Sadeghi, D. Doran, and M. Allahyari, "A broad evaluation of the tor english content ecosystem," arXiv preprint arXiv:1902.06680, Feb. 2019.

[4] J. Jin, N. Kim, S. Lee, and D. Jeong, "Forensic investigation of the dark web on the tor network: Pathway toward the surface web," International Journal of Information Security, 2023.

[5] M. Bugajewska, "A survey of challenges in dark web crawling: Technical, security, and ethical perspective," in Communications in Computer and Information Science. Springer, 2025, pp. 349–355.

[6] Y. Boshmaf, I. Perera, U. Kumarasinghe et al., "Dizzy: Largescale crawling and analysis of onion services," arXiv preprint arXiv:2209.07202, Sep. 2022.

[7] J. Pastor-Galindo, H.-A. Sandlin, F. G. Marmol, G. Bovet, and G. M. Perez, "A big data architecture for early identification and categorization of dark web

sites," Future Generation Computer Systems, vol. 157, pp. 67–81, Aug. 2024.

[8] D. De Pascale, G. Cascavilla, D. A. Tamburri, and W.-J. Van Den Heuvel, "Crator: A crawler for tor: Turning dark web pages into open source intelligence," in Proceedings of the 29th European Symposium on Research in Computer Security (ESORICS), ser. Lecture Notes in Computer Science, vol. 14983. Springer, 2024, pp. 144–161.

[9] Anonymous, "Illicit darkweb classification via natural-language processing: Classifying illicit content of webpages based on textual information," arXiv preprint arXiv:2312.04944, Dec. 2023.

[10] H. T. Luong, "Foundations and trends in the darknet-related criminals in the last 10 years: a systematic literature review and bibliometric analysis," Security Journal, vol. 37, no. 3, pp. 535–574, 2024.

[11] H. Khatter, N. Aggrawal, and A. K. Shrivastava, "Machine learning based sentiment analysis of tweets," in 2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT), 2023, pp. 148–152.

[12] P. Agrawal, A. Kumar, and A. K. Tripathi, "Mitigating cyberbullying in social media: A deep contextual learning approach for severity level classification in textual data," in 2024 5th International Conference on Electronics and Sustainable Communication Systems (ICESC), 2024, pp. 926–934.

[13] A. Rodriguez Barredo-Valenzuela, S. Pastrana, and G. Suarez-Tangil, "Snorkeling in dark waters: A longitudinal surface exploration of unique tor hidden services," arXiv preprint arXiv:2504.16836, Apr. 2025.

[14] N. Yazdanjue, M. Rakhshaninejad, H. Yazdanjouei, M. S. Khorshidi, M. S. Niemela, F. Chen, and A. H. Gandomi, "A language model-driven semi-supervised ensemble framework for illicit market detection across deep/dark web and social platforms," arXiv preprint arXiv:2507.22912, Jul. 2025.

[15] R. Singh, R. Kashyap, and V. Sharma, "Toxic comment analyzer using bert: A deep learning approach for toxicity detection," in 2023 Second International Conference on Informatics (ICI). IEEE, 2023, pp. 1–6